NUTRITION APP

Documentación Técnica del Proyecto

Aplicación de gestión nutricional

Técnicas de Programación ·Proyecto #1

Luis Esteban Vega Varela

Versión 1.0 · 2026

Índice

Índice 2
1. Introducción 3
1.1 Objetivos del proyecto 3
1.2 Tecnologías utilizadas 4
2. Decisiones de Diseño 4
2.1 Arquitectura MVC 4
2.2 Patrón Repository 4
2.3 Herencia y Polimorfismo 5
2.4 Almacenamiento en JSON 5
2.5 Manejo de errores 5
2.6 Decisión sobre Email en lugar de Username 5
3. Desarrollo 6
3.1 Estructura del proyecto 6
3.2 Requisitos para ejecutar el programa 6
3.3 Cómo ejecutar el programa 6
3.4 Flujo de la aplicación 7
3.5 Interacciones principales 12
4. Conclusiones 13

1. Introducción

NutritionApp es una aplicación de escritorio desarrollada en C# utilizando Windows Forms y la arquitectura MVC (Modelo-Vista-Controlador) cuyo propósito principal es ayudar a los usuarios a gestionar su alimentación diaria, monitorear su consumo nutricional y alcanzar sus objetivos de salud de manera organizada y personalizada.

El proyecto fue concebido y desarrollado a lo largo de cuatro sprints de trabajo bajo la metodología SCRUM, lo cual permitió organizar el desarrollo de forma incremental, entregando valor funcional al final de cada iteración. Desde el sprint inicial, que estableció las bases del sistema de autenticación de usuarios, hasta el sprint final con el módulo de estadísticas y reportes, cada etapa representó un avance significativo en la construcción de una solución completa y robusta.
Desde el punto de vista técnico, la aplicación implementa la arquitectura Modelo-Vista-Controlador (MVC) como columna vertebral del proyecto, garantizando una separación clara entre la lógica de negocio, el acceso a datos y la interfaz gráfica. Adicionalmente, se aplicó el patrón de diseño Repository para gestionar el acceso a los datos almacenados en archivos JSON, lo cual proporciona una capa de abstracción que facilita posibles migraciones tecnológicas en el futuro.

El resultado es una aplicación completamente funcional que ofrece registro e inicio de sesión de usuarios, cálculo automático de IMC y necesidades calóricas basado en la fórmula de Mifflin-St Jeor, gestión completa de menús diarios, un catálogo de 250 alimentos ordenado alfabéticamente, menús sugeridos personalizados según el tipo de dieta del usuario, y un módulo de estadísticas con desglose diario por rango de fechas. Todo esto con una interfaz sencilla, intuitiva y orientada a cualquier tipo de usuario, sin importar su nivel de experiencia tecnológica.

La presente documentación técnica describe en detalle las decisiones de diseño tomadas durante el desarrollo, el proceso de construcción del sistema, las instrucciones para ejecutar y utilizar la aplicación, y los aprendizajes obtenidos a lo largo del proyecto.

1.1 Objetivos del proyecto

- Aplicar la arquitectura MVC en una aplicación de escritorio real.
- Implementar los principios de programación orientada a objetos: herencia, polimorfismo y encapsulamiento.
- Aplicar el patrón de diseño Repository para el acceso a datos.
- Desarrollar una aplicación funcional que resuelva un problema real del usuario.
- Seguir buenas prácticas de Clean Code y principios SOLID.

1.2 Tecnologías utilizadas

Lenguaje	C#
Framework	.NET 10.0 Windows
Interfaz gráfica	Windows Forms
Almacenamiento	Archivos JSON
Librería JSON	Newtonsoft.Json 13.0.3
IDE	Visual Studio Code
Control de versiones	Git / GitHub
Gestión de proyecto	Jira (SCRUM)

2. Decisiones de Diseño

2.1 Arquitectura MVC

Se eligió la arquitectura MVC para separar claramente las responsabilidades del sistema. Esta separación facilita el mantenimiento del código y permite modificar una capa sin afectar las demás.

Capa	Responsabilidad	Archivos principales
Model	Define la estructura de los datos del sistema.	User.cs, Menu.cs, MenuItem.cs, FoodProduct.cs, NutritionPlan.cs
View	Maneja la interfaz gráfica y la interacción con el usuario.	LoginForm, DashboardForm, MenuListForm, MenuEditForm, StatisticsForm
Controller	Contiene la lógica de negocio y conecta Models con Views.	UserController, MenuController, FoodController, SuggestedMenuController

2.2 Patrón Repository

Se implementó el patrón Repository para separar la lógica de acceso a datos de la lógica de negocio. Cada entidad tiene su propio repositorio independiente, lo cual facilita la escalabilidad y el mantenimiento.

UserRepository	Lee y guarda usuarios en users.json
MenuRepository	Lee y guarda menús en menus.json
FoodRepository	Lee el catálogo de alimentos desde foods.json

Esta decisión permite que, si en el futuro se cambia el almacenamiento de JSON a una base de datos, solo se necesita modificar los repositorios sin tocar los controladores ni las vistas.

2.3 Herencia y Polimorfismo

Se implementó una jerarquía de clases para los planes nutricionales. La clase abstracta NutritionPlan define el contrato que todas las dietas deben cumplir, y cada subclase implementa sus propios cálculos de macronutrientes.

Clase	Tipo	Proteína	Carbohidratos	Grasas
NutritionPlan	Abstracta (base)	-	-	-
StandardPlan	Concreta	30%	40%	30%
KetoPlan	Concreta	25%	5%	70%
VegetarianPlan	Concreta	20%	55%	25%

Gracias al polimorfismo, el NutritionService puede llamar GetProteinGrams(), GetCarbGrams() y GetFatGrams() sin saber qué tipo de plan es, y cada subclase calcula los valores correctamente.

2.4 Almacenamiento en JSON

Se eligió JSON como mecanismo de almacenamiento por su simplicidad y facilidad. Los archivos se copian automáticamente al directorio de salida mediante la configuración del archivo .csproj.

users.json	25 usuarios con diferentes perfiles, dietas y objetivos.
foods.json	250 alimentos ordenados alfabéticamente con sus macros por porción.
menus.json	1500 registros de menús (25 usuarios × 60 días × 4 comidas).

2.5 Manejo de errores

Se implementó manejo de excepciones con bloques try/catch en todos los repositorios y controladores. Los errores se muestran al usuario mediante MessageBox para mantener la simplicidad.

2.6 Email

Se decidió usar el correo electrónico como identificador único del usuario en lugar de un nombre de usuario. Esto garantiza unicidad y es más cercano a cómo funcionan las aplicaciones reales.

3. Desarrollo

3.1 Estructura del proyecto

El proyecto sigue una estructura de carpetas organizada por capas de la arquitectura MVC:

Carpeta	Contenido
Models/	Clases de datos: User, Menu, MenuItem, FoodProduct, NutritionPlan y subclases, MacroDistribution.
Views/	Formularios de Windows Forms: Login, Register, Dashboard, MenuList, MenuEdit, EditProfile, Statistics, SuggestedMenus.
Controllers/	Lógica de negocio: UserController, MenuController, FoodController, SuggestedMenuController.
Data/	Repositorios y rutas de archivos: UserRepository, MenuRepository, FoodRepository, DataPaths.
Services/	Cálculos nutricionales: NutritionService.
Data/Files/	Archivos JSON de datos: users.json, foods.json, menus.json.

3.2 Requisitos para ejecutar el programa

Sistema operativo	Windows 10 o superior
.NET SDK	.NET 10.0 o superior
IDE recomendado	Visual Studio Code o Visual Studio 2022
Extensión VS Code	C# Dev Kit

3.3 Cómo ejecutar el programa

Desde Visual Studio Code

1. Abre la carpeta del proyecto en VS Code con File → Open Folder.
2. Abre una terminal desde el menú principal .
3. Ejecuta el comando: dotnet build
4. Si no hay errores, ejecuta: dotnet run
5. La aplicación se abrirá con la pantalla de Login.

Credenciales de prueba

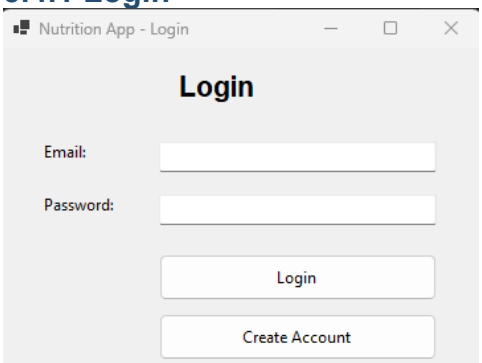
Email	john@email.com
Contraseña	1234

3.4 Flujo de la aplicación

La aplicación sigue el siguiente flujo de navegación entre pantallas:

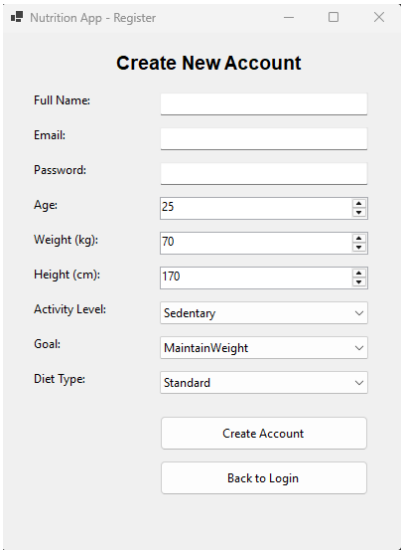
Pantalla	Acceso desde	Acciones disponibles
Login	Inicio de la aplicación	Iniciar sesión, ir a Registro
Register	Botón Create Account en Login	Crear cuenta, volver a Login
Dashboard	Login exitoso	Ver info nutricional, ir a Menús, Estadísticas, Editar Perfil, Logout
My Menus	Botón My Menus en Dashboard	Ver, crear, editar, eliminar menús, ver menús sugeridos
New Menu	Botón New Menu en My Menus	Crear un nuevo menú
Edit Menu	Botones New Menu / Edit Menu en My Menus	Agregar/quitar alimentos, calcular totales, guardar
Delete Menu	Botón Deleted Menu en My Menus	Eliminar los menús agregados
Suggested Menus	Botón Suggested Menus en My Menus	Ver menús predefinidos por dieta, agregar a mis menús
Edit Profile	Botón Edit Profile en Dashboard	Modificar datos del perfil, guardar cambios
Statistics	Botón Statistics en Dashboard	Seleccionar rango de fechas, ver reportes y desglose diario

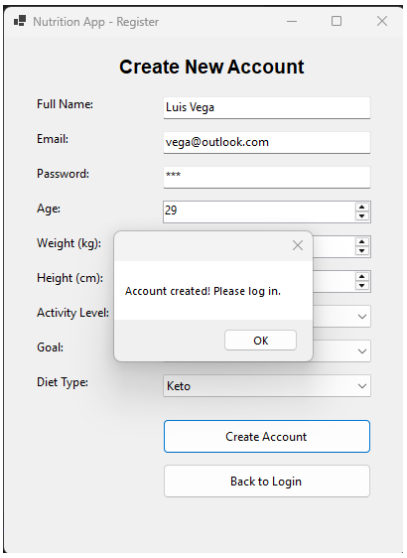
3.4.1 Login



Esta es la pantalla principal del programa, en ella dispondremos de dos opciones, hacer login a nuestra cuenta de usuario o registrarnos como un nuevo usuario en NutritionApp.

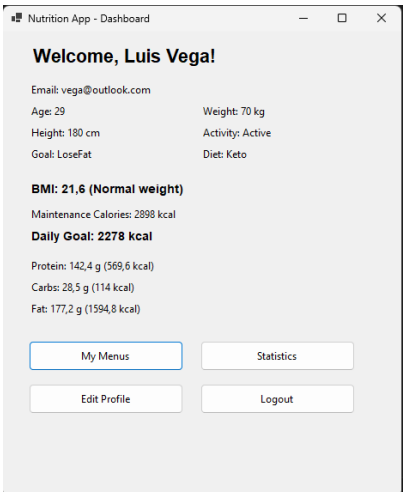
3.4.2 Registro





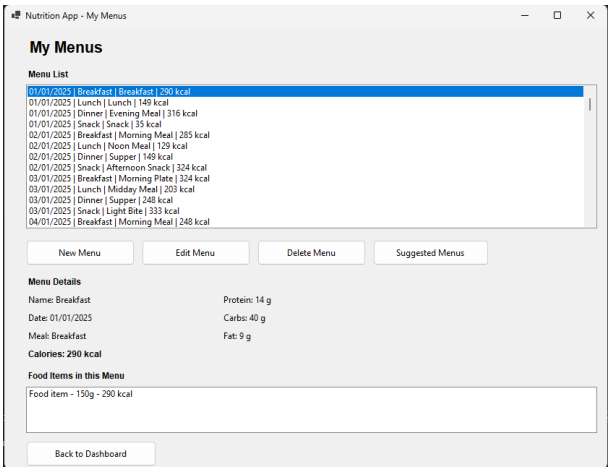
Acá es donde los nuevos usuarios van a colocar los datos necesarios para la creación de su cuenta. Después de esto, se mostrará una pantalla indicando que la reacción fue exitosa y nos regresará a la pantalla principal para logearnos en el programa.

3.4.3 Dashboard



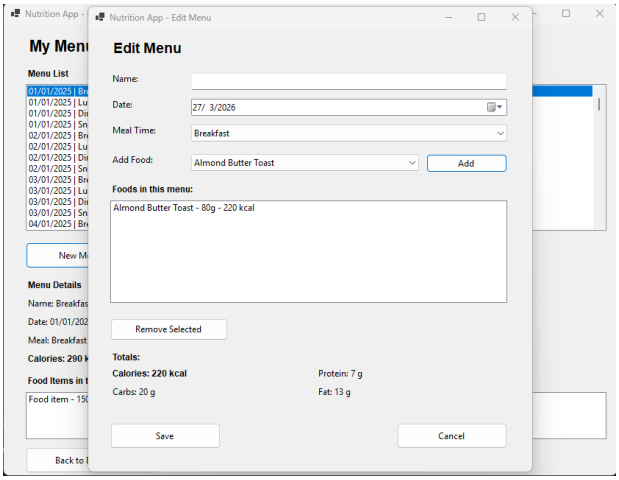
El dashboard es la pantalla principal de la cuenta del usuario, acá va a poder visualizar sus datos y desplazarse por las diferentes opciones del dashboard.

3.4.4 My Menus



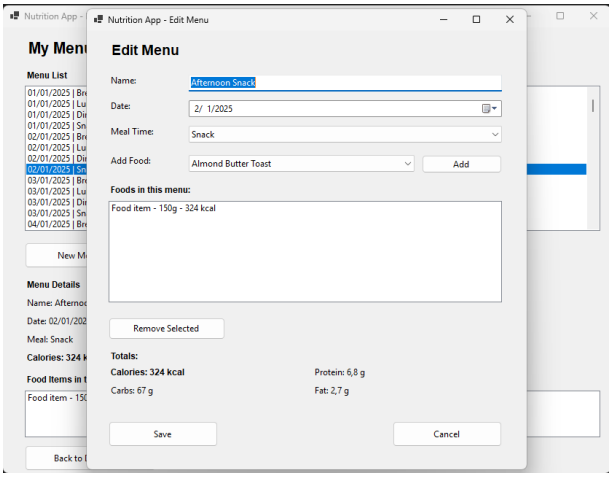
En esta sección se mostrarán los menús agregados por el usuario, junto con el total calórico de cada uno, información de este. Además, se muestran las diferentes opciones dentro de esta pantalla.

3.4.5 New Menu



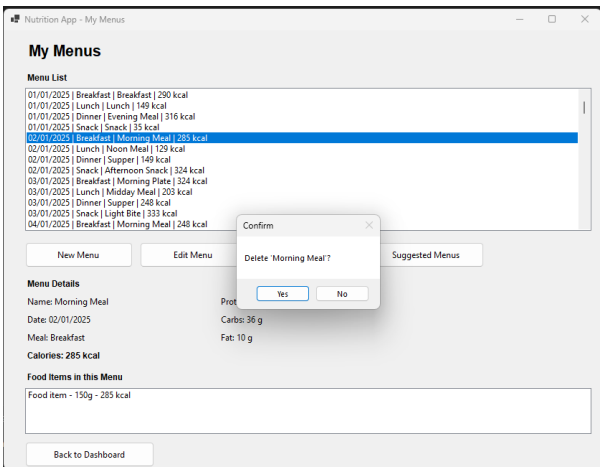
La opción de *New Menu* nos va a permitir crear menús, gracias a la gran variedad de opciones de alimentos con las que cuenta el programa, le permite al usuario una gran variedad de opciones para la creación de menú. De igual forma, vamos a poder observar la información de cada alimento que seleccionemos.

3.4.6 Edit Menu



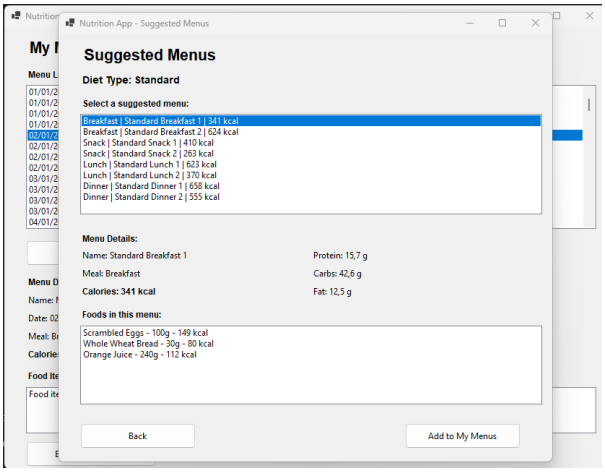
De la misma forma, edit menu, nos permite modificar menús ya creados y guardados en nuestro perfil.

3.4.7 Delete Menu



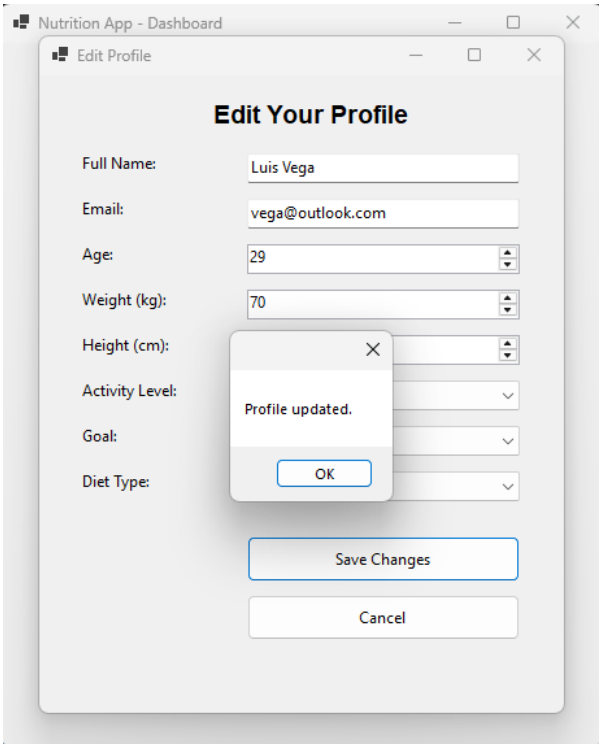
Delete menú nos va a permitir eliminar menús que se encuentren en nuestro perfil de usuario.

3.4.8 Suggested Menus



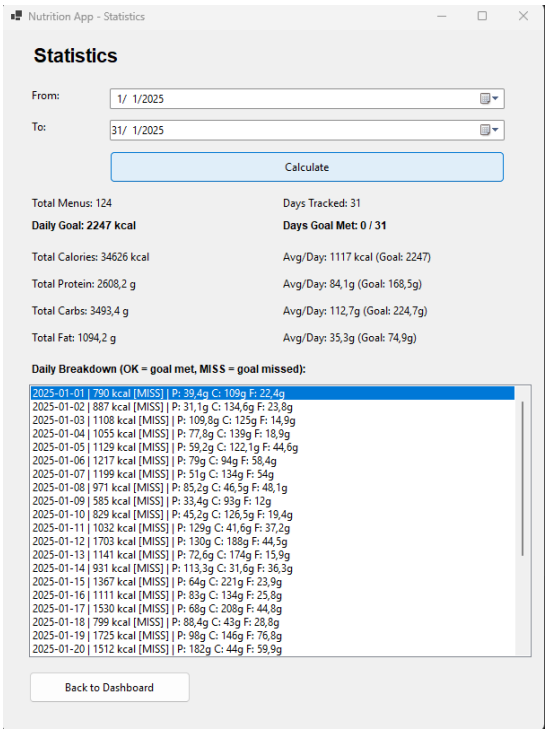
En la sección de suggested menus, podemos encontrar menús sugeridos según la dieta que elegimos previamente y brindándonos la información de como esta compuesto el menú.

3.4.9 Edit Profile



En esta sección el usuario va a poder editar los datos de su perfil. Mostrando un aviso de que los camios fuera modificados al presionar el botón de salvar cambios.

3.4.10 Statistics



En esta sección vamos a poder visualizar las estadísticas del proceso, según el rango de fecha que queramos y así ver los menús que nos realizamos por día y ver si logramos o no la meta calórica acorde a nuestro menú.

3.5 Interacciones principales

Registro de usuario

El usuario completa el formulario con sus datos personales, nivel de actividad, objetivo nutricional y tipo de dieta. Al guardar, el sistema verifica que el email no exista ya y asigna un ID único. Los datos se guardan en users.json.

Cálculo nutricional en el Dashboard

Al iniciar sesión, el Dashboard calcula automáticamente el IMC, las calorías de mantenimiento usando la fórmula de Mifflin-St Jeor, y las calorías diarias ajustadas según el objetivo y tipo de dieta del usuario. Los macronutrientes se calculan usando la clase correspondiente según el tipo de dieta (StandardPlan, KetoPlan o VegetarianPlan) gracias al polimorfismo.

Gestión de menús

El usuario puede crear menús asociados a una fecha y tipo de comida (Breakfast, Lunch, Dinner, Snack). Cada menú puede contener múltiples alimentos del catálogo de 250 productos. Los totales de calorías y macronutrientes se recalculan automáticamente cada vez que se agrega o elimina un alimento.

Menús sugeridos

La aplicación genera 8 menús predefinidos según el tipo de dieta del usuario (2 por cada tipo de comida). El usuario puede revisar los alimentos y valores nutricionales de cada menú sugerido y agregarlo directamente a su lista de menús.

Estadísticas

El usuario selecciona un rango de fechas y la aplicación calcula los totales y promedios de calorías y macronutrientes para ese período. Por cada día se muestra si se cumplió la meta calórica (OK) o no (MISS), con una tolerancia de ±100 kcal. Los datos de prueba cubren 60 días por usuario.

4. Conclusiones

El desarrollo de NutritionApp permitió aplicar de forma práctica los conceptos teóricos del curso. La combinación de arquitectura MVC, patrón Repository, herencia y polimorfismo resultó en una aplicación bien estructurada, mantenible y escalable.

El proyecto demostró que seguir principios de diseño desde el inicio del desarrollo resulta en un código más limpio y fácil de extender. La adición de nuevas funcionalidades como los Menús Sugeridos o nuevos tipos de dieta puede hacerse sin modificar el código existente gracias a las decisiones de diseño tomadas.

Trabajar con SCRUM y Jira aportó una visión más clara sobre cómo organizar el desarrollo de software en etapas. Dividir el proyecto en cuatro sprints permitió enfocarse en una funcionalidad a la vez, detectar problemas de forma temprana y entregar avances concretos al final de cada semana. Esta forma de trabajar resultó más efectiva que intentar construir todo al mismo tiempo.

A lo largo del desarrollo también se presentaron dificultades que obligaron a investigar y buscar soluciones fuera de lo visto en clase. Superar esos obstáculos fue parte importante del aprendizaje y dejó una base más sólida para enfrentar proyectos futuros.

En términos generales, NutritionApp cumplió con todos los requerimientos planteados al inicio y el proceso de desarrollo dejó aprendizajes concretos tanto a nivel técnico como a nivel de organización y planificación. El proyecto es una muestra de que aplicar buenas prácticas desde el principio, aunque requiere más esfuerzo inicial, hace que el resultado final sea de mayor calidad y más fácil de mantener.